```
pip install openpyxl
```

```
Collecting openpyxl
  Downloading openpyxl-3.1.5-py2.py3-none-any.whl.metadata (2.5 kB)
Collecting et-xmlfile (from openpyxl)
  Downloading et_xmlfile-2.0.0-py3-none-any.whl.metadata (2.7 kB)
Downloading openpyxl-3.1.5-py2.py3-none-any.whl (250 kB)
Downloading et_xmlfile-2.0.0-py3-none-any.whl (18 kB)
Installing collected packages: et-xmlfile, openpyxl
Successfully installed et-xmlfile-2.0.0 openpyxl-3.1.5
Note: you may need to restart the kernel to use updated packages.
```

```
import pandas as pd

# Load file
file_path = "Suggestions YTD 6-11.xlsx"
df = pd.read_excel(file_path, sheet_name="in")
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 154702 entries, 0 to 154701
Data columns (total 12 columns):
 #   Column                       Non-Null Count   Dtype
---  ------                       --------------   -----
 0   Record Type                  154702 non-null  object
 1   Posted Date                  154702 non-null  datetime64[ns]
 2   Expiration Date              154702 non-null  datetime64[ns]
 3   Actioned                     99983 non-null   float64
 4   Dismissed                    27143 non-null   float64
 5   Marked As Complete           7987 non-null    float64
 6   Title                        154702 non-null  object
 7   Reason                       154702 non-null  object
 8   Last Modified Date           154702 non-null  datetime64[ns]
 9   Last Modified By: Full Name  154702 non-null  object
 10  Owner: Full Name             154702 non-null  object
 11  Account: Name                154702 non-null  object
dtypes: datetime64[ns](3), float64(3), object(6)
memory usage: 14.2+ MB
```

```
# Convert Date Columns to Datetime
date_cols = ['Posted Date', 'Expiration Date', 'Last Modified Date']
for col in date_cols:
    df[col]=pd.to_datetime(df[col], errors='coerce')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 154702 entries, 0 to 154701
Data columns (total 12 columns):
 #   Column                       Non-Null Count   Dtype
---  ------                       --------------   -----
 0   Record Type                  154702 non-null  object
 1   Posted Date                  154702 non-null  datetime64[ns]
 2   Expiration Date              154702 non-null  datetime64[ns]
```

```
 3   Actioned                     99983 non-null   float64
 4   Dismissed                    27143 non-null   float64
 5   Marked As Complete           7987 non-null    float64
 6   Title                        154702 non-null  object
 7   Reason                       154702 non-null  object
 8   Last Modified Date           154702 non-null  datetime64[ns]
 9   Last Modified By: Full Name  154702 non-null  object
 10  Owner: Full Name             154702 non-null  object
 11  Account: Name                154702 non-null  object
dtypes: datetime64[ns](3), float64(3), object(6)
memory usage: 14.2+ MB
```

```
# Clean HTML from the Reason Field
from bs4 import BeautifulSoup

df['Reason_clean'] = df['Reason'].apply(lambda x: BeautifulSoup(x, "html.parser").get_text() if isinstance(x, str) else "")
```

```
df.head(5)
```

	Record Type	Posted Date	Expiration Date	Actioned	Dismissed	Marked As Complete	Title	Reason	Last Modified Date	Last Modified By: Full Name	Owner: Full Name	Accou Name
0	Schedule Call	2025-04-06	2025-04-12	1.0	NaN	NaN	High CGM Growth	<ul><li>Adriana Cadena-Martinez's CGM TRx has ...	2025-04-13	Aktana Integration	Abigail Rosales	Adrian Caden Martin
1	Schedule Call	2025-02-23	2025-03-01	1.0	NaN	NaN	Leading FSL TRx Growth with respect to Goal	<ul><li>Adriana Cadena-Martinez's FSL TRx are ...	2025-03-02	Aktana Integration	Abigail Rosales	Adrian Caden Martin
2	Schedule Call	2025-02-02	2025-02-08	1.0	NaN	NaN	HCP Dexcom Adoption for Conventional Insulin (...	<ul><li>Adriana Cadena-Martinez?s FSL TRx Shar...	2025-02-09	Aktana Integration	Abigail Rosales	Adrian Caden Martin
3	Schedule Call	2025-05-25	2025-05-31	1.0	NaN	NaN	Lagging Product Transition	<ul><li>Alberto Almeida's FSL 2+ and 3+ TRx vo...	2025-06-01	Aktana Integration	Abigail Rosales	Alberto Almeid
4	Schedule Call	2025-01-12	2025-01-18	1.0	NaN	NaN	HCP scripts are not being filled	<ul><li>Alberto Almeida's new patients have a ...	2025-01-19	Aktana Integration	Abigail Rosales	Alberto Almeid

```
# Fill Missing Values in numeric Fields
na_per_col = df.isna().sum()
print(na_per_col)
```

```
Record Type                         0
Posted Date                         0
Expiration Date                     0
Actioned                        54719
Dismissed                      127559
Marked As Complete             146715
Title                               0
Reason                              0
Last Modified Date                  0
Last Modified By: Full Name         0
Owner: Full Name                    0
Account: Name                       0
Reason_clean                        0
dtype: int64
```

```
df['Actioned'] = df['Actioned'].fillna(0)
df['Dismissed'] = df['Dismissed'].fillna(0)
df['Marked As Complete'] = df['Marked As Complete'].fillna(0)
```

```
na_per_col = df.isna().sum()
print(na_per_col)
```

```
Record Type                    0
Posted Date                    0
Expiration Date                0
Actioned                       0
Dismissed                      0
Marked As Complete             0
Title                          0
Reason                         0
Last Modified Date             0
Last Modified By: Full Name    0
Owner: Full Name               0
Account: Name                  0
Reason_clean                   0
dtype: int64
```

```
# Create derived columns
df['Days_To_Expire'] = (df['Expiration Date'] - df['Posted Date']).dt.days
df['Days_To_Modify'] = (df['Last Modified Date'] - df['Posted Date']).dt.days
df['Was_Actioned'] = df['Actioned'] > 0
```

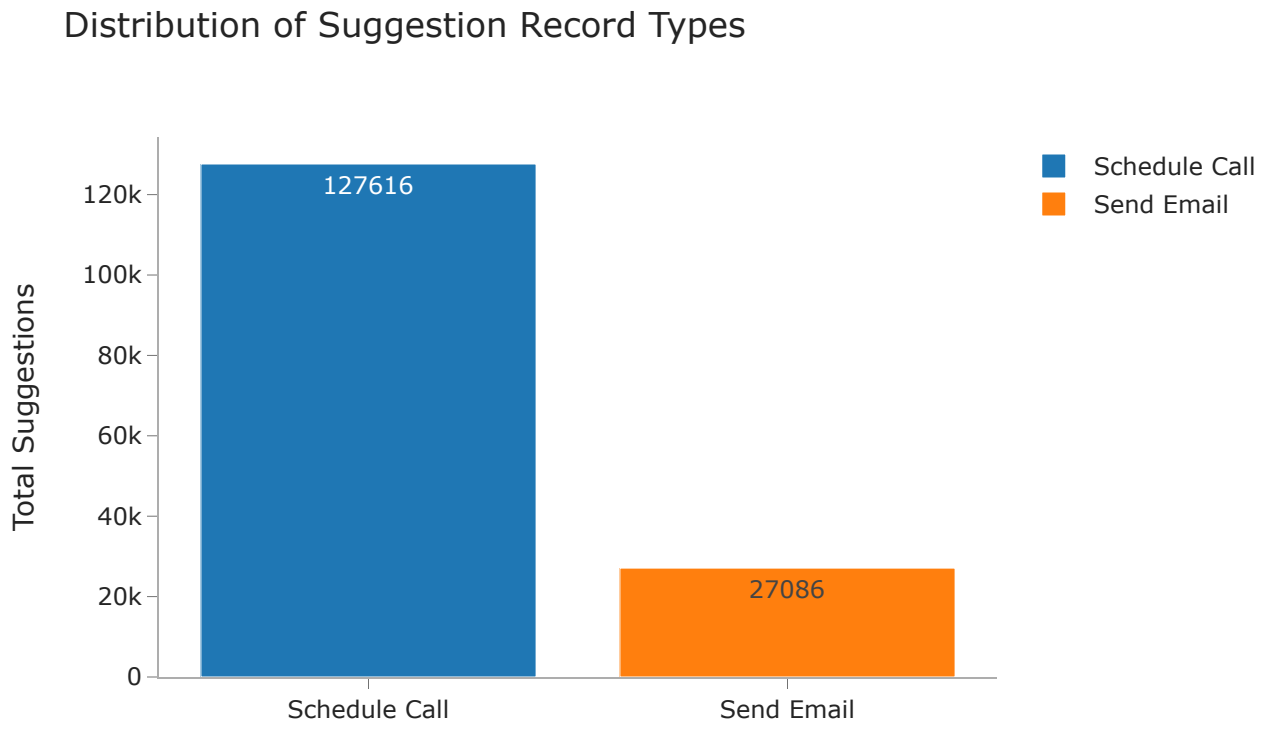
```
df.head(3)
```

	Record Type	Posted Date	Expiration Date	Actioned	Dismissed	Marked As Complete	Title	Reason	Last Modified Date	Last Modified By: Full Name	Owner: Full Name	Accou Name
0	Schedule Call	2025-04-06	2025-04-12	1.0	0.0	0.0	High CGM Growth	<ul><li>Adriana Cadena-Martinez's CGM TRx has ...	2025-04-13	Aktana Integration	Abigail Rosales	Adrian Caden Martin
1	Schedule Call	2025-02-23	2025-03-01	1.0	0.0	0.0	Leading FSL TRx Growth with respect to Goal	<ul><li>Adriana Cadena-Martinez's FSL TRx are ...	2025-03-02	Aktana Integration	Abigail Rosales	Adrian Caden Martin
2	Schedule Call	2025-02-02	2025-02-08	1.0	0.0	0.0	HCP Dexcom Adoption for Conventional Insulin (...	<ul><li>Adriana Cadena-Martinez?s FSL TRx Shar...	2025-02-09	Aktana Integration	Abigail Rosales	Adrian Caden Martin

EDA

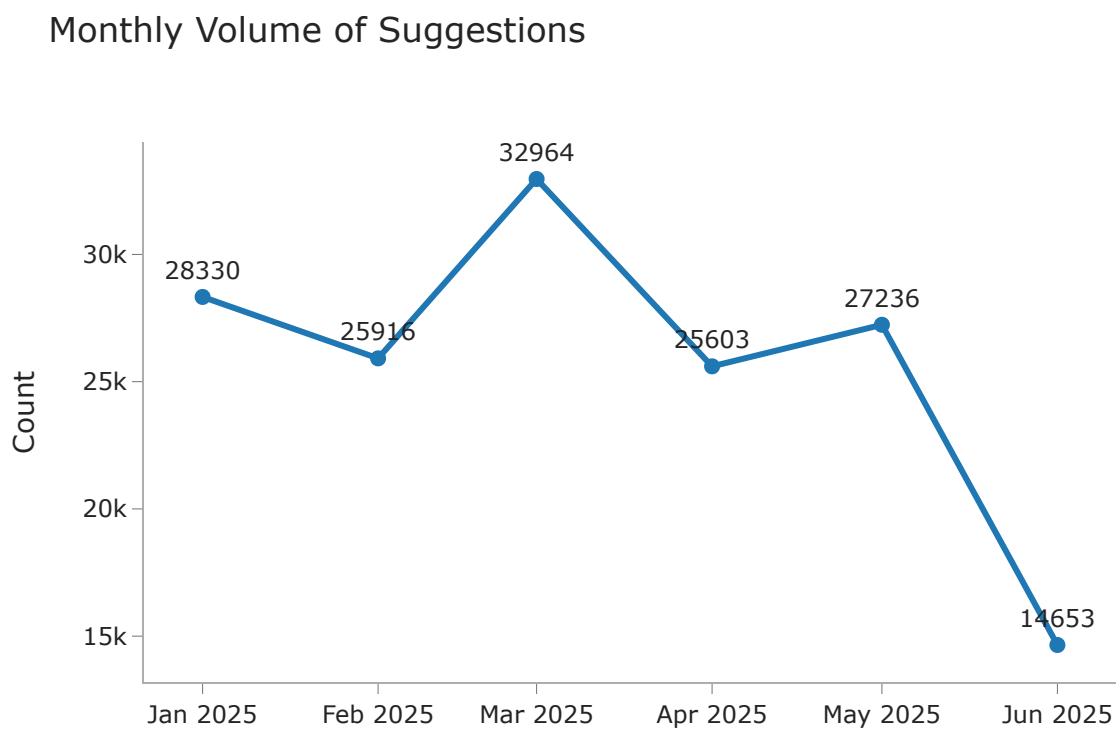
```
import plotly.express as px
```

```
# 1. Record Type Distribution

# Recompute record type counts properly
record_counts_df = df['Record Type'].value_counts().reset_index()
record_counts_df.columns = ['Record Type', 'Count']  # Rename columns

fig1 = px.bar(
    record_counts_df.sort_values('Count', ascending=False),
    x='Record Type',
    y='Count',
    text='Count',
    color='Record Type',
    title='Distribution of Suggestion Record Types',
    labels={'Record Type': '', 'Count': 'Total Suggestions'},
    template='simple_white'
)

fig1.show()
```

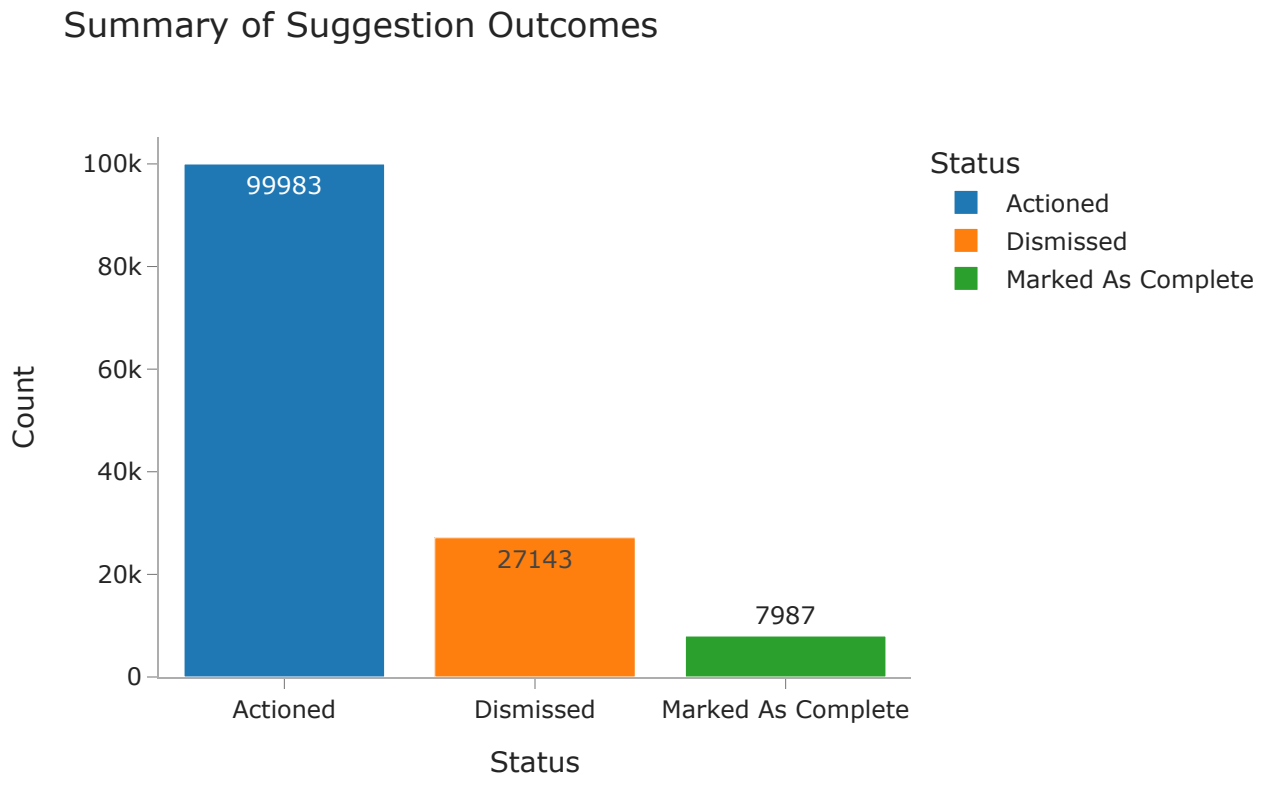



```
# 2. Suggestions Over Time

df['Posted Month'] = df['Posted Date'].dt.to_period('M').astype(str)
monthly_counts = df.groupby('Posted Month').size().reset_index(name='Count')

fig2 = px.line(
    monthly_counts,
    x='Posted Month',
    y='Count',
    text='Count',  # Show values
    markers=True,
    title='Monthly Volume of Suggestions',
    labels={'Count': 'Total Suggestions'},
    template='simple_white'
)

fig2.update_traces(
    line=dict(width=3),
    marker=dict(size=8),
    textposition='top center',  # Label position
    textfont=dict(size=12)
)

fig2.update_layout(
    xaxis_title='',
    yaxis_title='Count',
    hovermode='x unified'
```

```
)


fig2.show()
```

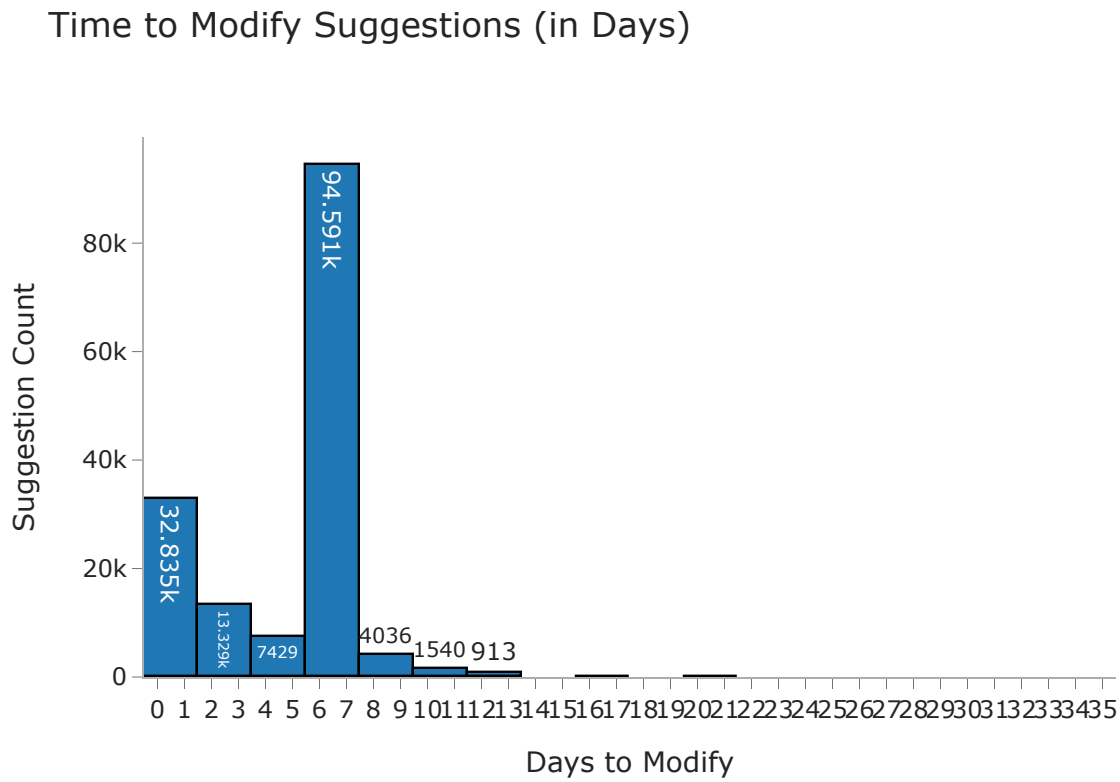



```
# 3. Suggestion Status Overview
status_df = pd.DataFrame({
    'Status': ['Marked As Complete', 'Actioned', 'Dismissed'],
    'Count': [
        df['Marked As Complete'].sum(),
        df['Actioned'].sum(),
        df['Dismissed'].sum()
    ]
})

fig3 = px.bar(
    status_df.sort_values('Count', ascending=False),
    x='Status',
    y='Count',
    text='Count',
    color='Status',
    title='Summary of Suggestion Outcomes',
    template='simple_white'
)


fig3.show()
```



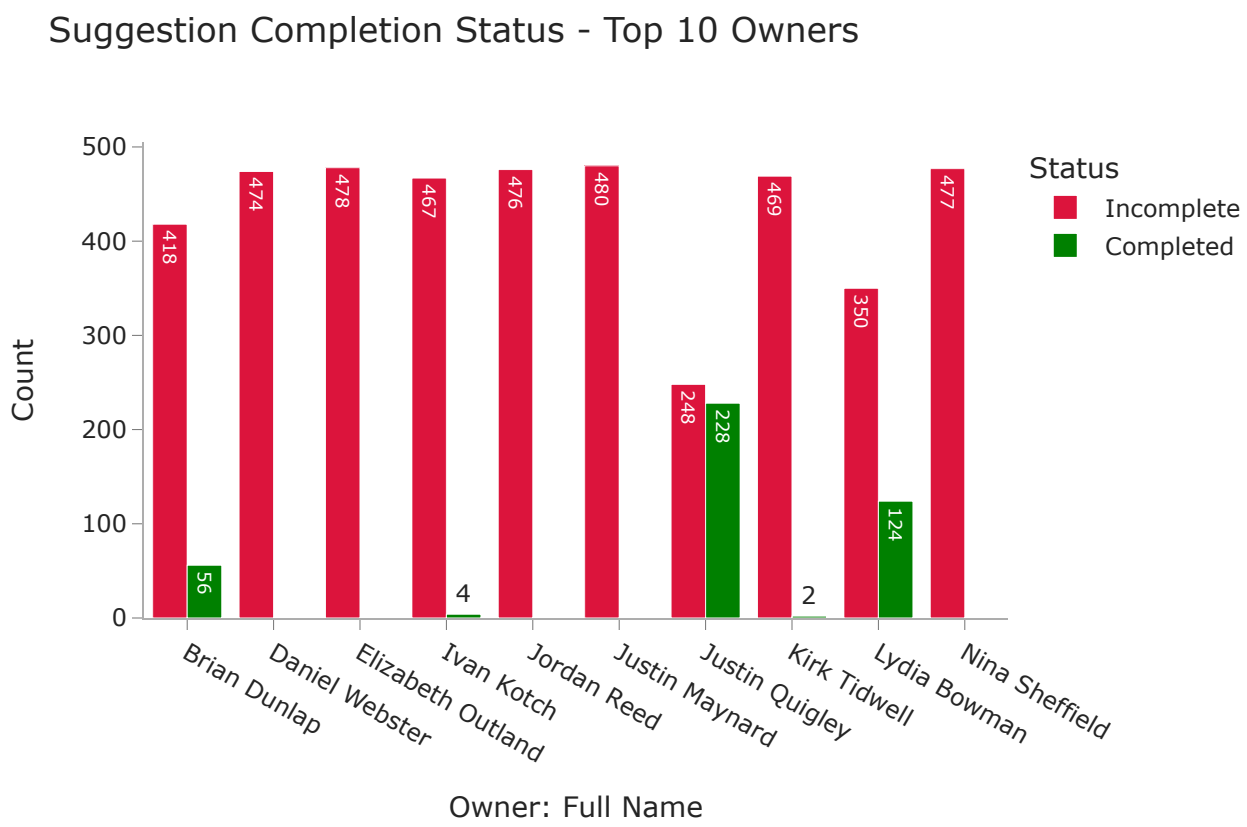

```
# 4. Days to Modify Suggestions
filtered_df = df[df['Days_To_Modify'] >= 0]

fig5 = px.histogram(
    filtered_df,
    x='Days_To_Modify',
    nbins=20,  # Less cluttered binning
    title='Time to Modify Suggestions (in Days)',
    labels={'Days_To_Modify': 'Days'},
    template='simple_white',
    text_auto=True  # Show bar labels automatically
)

fig5.update_traces(
    marker_line_color='black',
    marker_line_width=1.2,
    textfont_size=12
)

fig5.update_layout(
    yaxis_title='Suggestion Count',
    xaxis_title='Days to Modify',
    xaxis=dict(
        tickmode='linear',
        tick0=0,
        dtick=1,
        tickangle=0
```

```
    )
)

fig5.show()
```



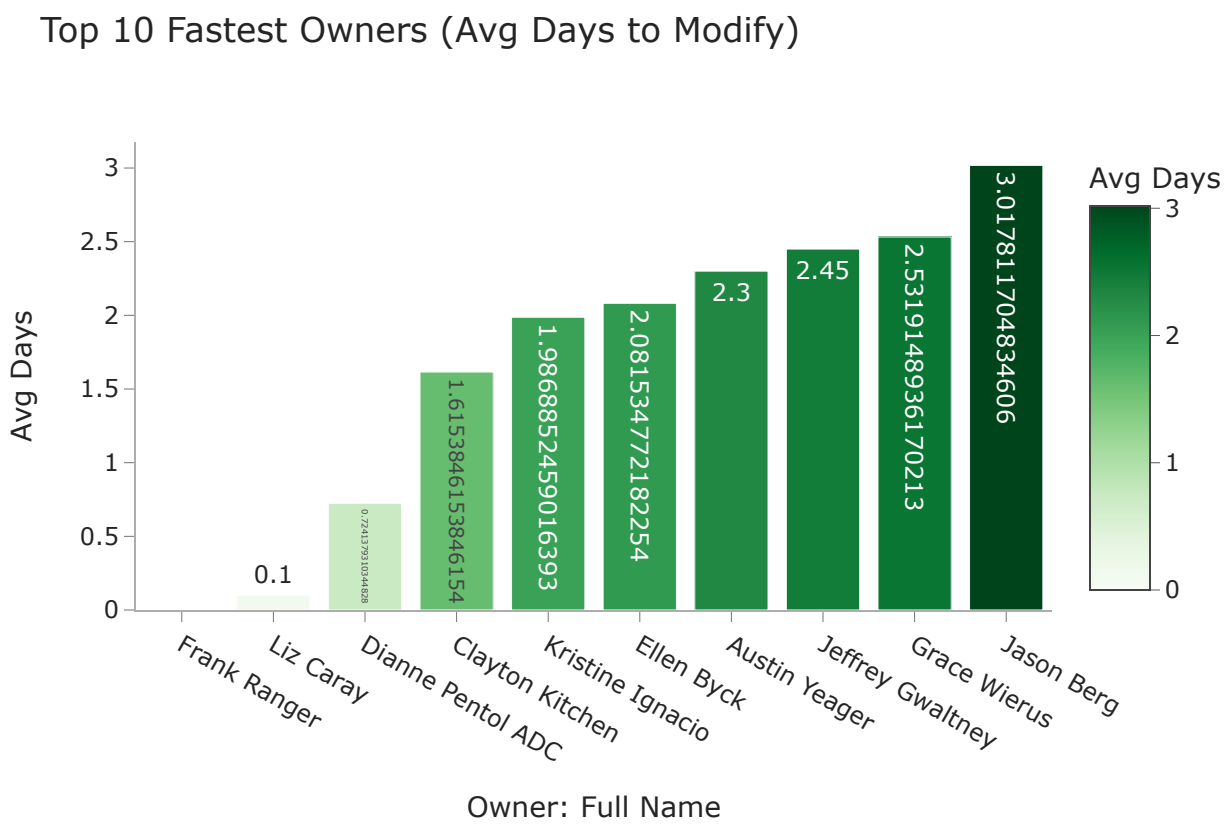

```
# 1. Volume vs. Completion by Owner
volume_df = (
    df.groupby(['Owner: Full Name', 'Marked As Complete'])
    .size().reset_index(name='Count')
)

# Map values for legend clarity
volume_df['Status'] = volume_df['Marked As Complete'].map({1: 'Completed', 0: 'Incomple
te'})

top_owners = df['Owner: Full Name'].value_counts().head(10).index
volume_df = volume_df[volume_df['Owner: Full Name'].isin(top_owners)]

fig = px.bar(
    volume_df,
    x='Owner: Full Name', y='Count', color='Status',
    color_discrete_map={'Completed': 'green', 'Incomplete': 'crimson'},
    barmode='group',
    text='Count',
    title='Suggestion Completion Status - Top 10 Owners',
    template='simple_white'
```

```
)


fig.show()
```

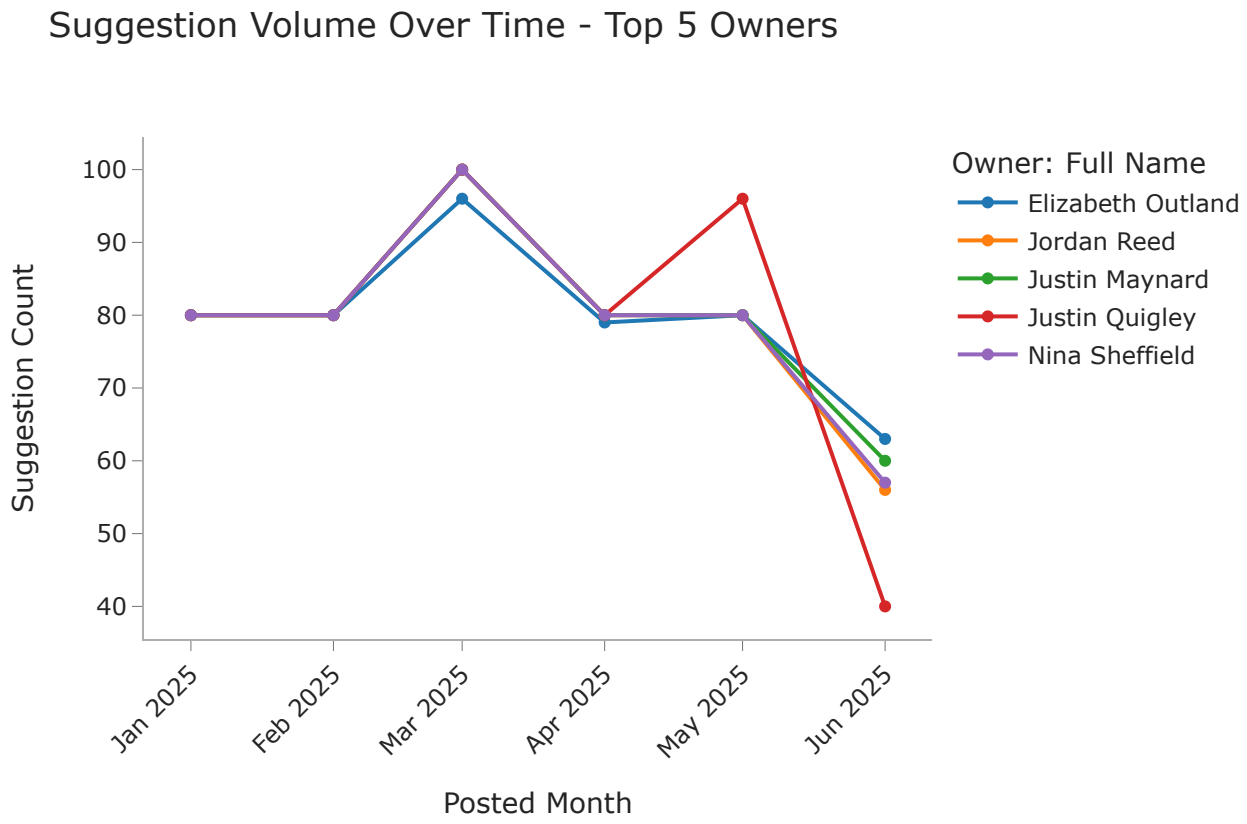



```
# 2. Box Plot: Time to Modify Suggestions by Owner
# Shows how fast each rep responds
owner_response = (
    df[df['Days_To_Modify'] >= 0]
    .groupby('Owner: Full Name')
    .agg(Average_Modify_Days=('Days_To_Modify', 'mean'), Count=
('Days_To_Modify', 'size'))
    .query('Count >= 10')  # remove outliers with few suggestions
    .sort_values('Average_Modify_Days')
    .head(10)
    .reset_index()
)

fig = px.bar(
    owner_response,
    x='Owner: Full Name', y='Average_Modify_Days',
    title='Top 10 Fastest Owners (Avg Days to Modify)',
    text='Average_Modify_Days',
    labels={'Average_Modify_Days': 'Avg Days'},
    template='simple_white',
    color='Average_Modify_Days',
    color_continuous_scale='greens'
```

```
)


fig.show()
```



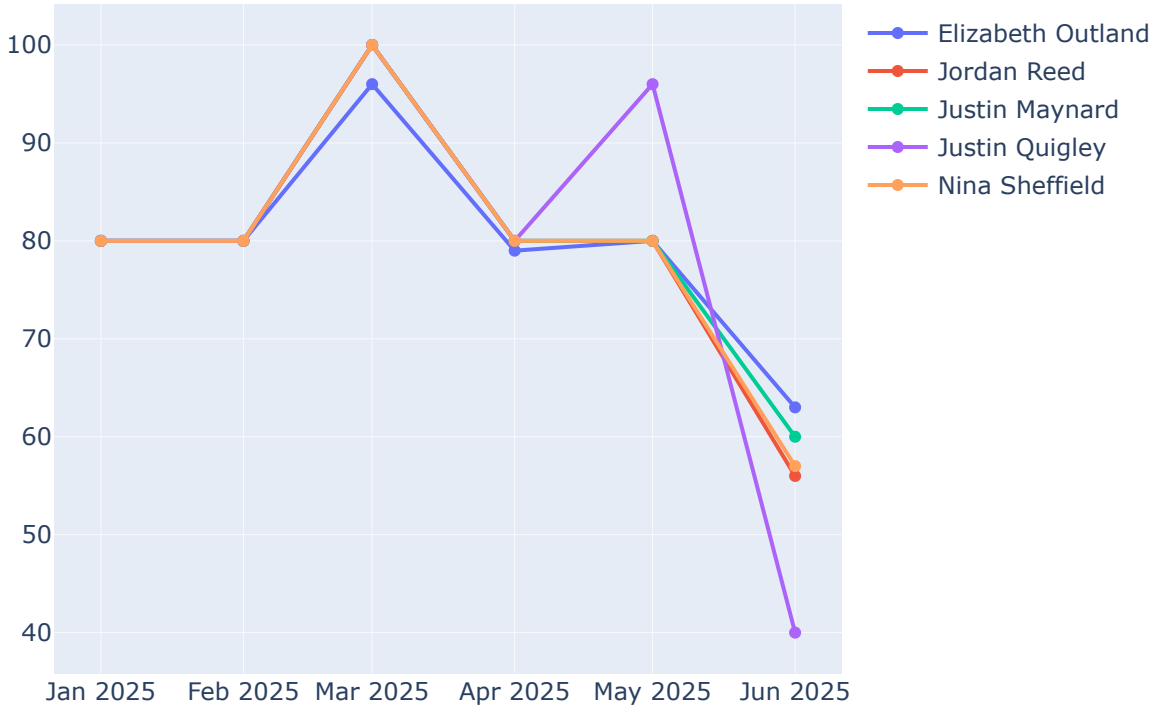

```
# 3.  Monthly Trend of Suggestion Volume for Top 5 Owners
top_owners = df['Owner: Full Name'].value_counts().head(5).index
df['Posted Month'] = df['Posted Date'].dt.to_period('M').astype(str)

monthly_owner_trend = df[df['Owner: Full Name'].isin(top_owners)] \
    .groupby(['Posted Month', 'Owner: Full Name']) \
    .size().reset_index(name='Suggestion Count')

fig = px.line(
    monthly_owner_trend,
    x='Posted Month', y='Suggestion Count',
    color='Owner: Full Name',
    markers=True,
    title='Suggestion Volume Over Time - Top 5 Owners',
    template='simple_white'
)
fig.update_layout(xaxis_tickangle=-45)

fig.show()
```




```
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Create subplot with 2 rows (chart + table)
fig = make_subplots(
    rows=2, cols=1,
    shared_xaxes=True,
    row_heights=[0.6, 0.4],
    vertical_spacing=0.1,
    specs=[[{"type": "xy"}], [{"type": "table"}]]
)

# Add line traces for each owner
owners = monthly_owner_trend['Owner: Full Name'].unique()
for owner in owners:
    owner_df = monthly_owner_trend[monthly_owner_trend['Owner: Full Name'] == owner]
    fig.add_trace(
        go.Scatter(
            x=owner_df['Posted Month'],
            y=owner_df['Suggestion Count'],
            mode='lines+markers',
            name=owner
        ),
        row=1, col=1
    )

# Create table
```

```
table_data = monthly_owner_trend.pivot(index='Owner: Full Name', columns='Posted Month'
, values='Suggestion Count').fillna(0).astype(int)
fig.add_trace(
    go.Table(
        header=dict(
            values=["Owner"] + list(table_data.columns),
            fill_color='paleturquoise',
            align='left'
        ),
        cells=dict(
            values=
[table_data.index] + [table_data[col] for col in table_data.columns],
            fill_color='lavender',
            align='left'
        )
    ),
    row=2, col=1
)

# Final layout
fig.update_layout(
    height=800,
    title_text="Suggestion Volume Over Time + Table",
    showlegend=True
)


fig.show()
```

Suggestion Volume Over Time + Table



Owner	2025-01	2025-02	2025-03	2025-04	2025-05	2025-06
Elizabet Outland	80	80	96	79	80	63
Jordan Reed	80	80	100	80	80	56
Justin Maynard	80	80	100	80	80	60
Justin Quigley	80	80	100	80	96	40

```
!pip install wordcloud
Collecting wordcloud
  Downloading wordcloud-1.9.4-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
Requirement already satisfied: numpy>=1.6.1 in /opt/python/envs/default_3_11/lib/python3
Requirement already satisfied: pillow in /opt/python/envs/default_3_11/lib/python3.11/si
Requirement already satisfied: matplotlib in /opt/python/envs/default_3_11/lib/python3.1
Requirement already satisfied: contourpy>=1.0.1 in /opt/python/envs/default_3_11/lib/pyt
Requirement already satisfied: cycler>=0.10 in /opt/python/envs/default_3_11/lib/python3
```

```
Requirement already satisfied: fonttools>=4.22.0 in /opt/python/envs/default_3_11/lib/py
Requirement already satisfied: kiwisolver>=1.3.1 in /opt/python/envs/default_3_11/lib/py
Requirement already satisfied: packaging>=20.0 in /opt/python/envs/default_3_11/lib/pyth
Requirement already satisfied: pyparsing>=2.3.1 in /opt/python/envs/default_3_11/lib/pyt
Requirement already satisfied: python-dateutil>=2.7 in /opt/python/envs/default_3_11/lib
Requirement already satisfied: six>=1.5 in /opt/python/envs/default_3_11/lib/python3.11/
Downloading wordcloud-1.9.4-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 547.9/547.9 kB 28.2 MB/s eta 0:00:00
Installing collected packages: wordcloud
Successfully installed wordcloud-1.9.4
```

```python
from wordcloud import WordCloud
import matplotlib.pyplot as plt

text_blob = ' '.join(df['Reason_clean'].dropna().astype(str))

wordcloud = WordCloud(width=1000, height=400, background_color='white').generate(text_b
lob)

plt.figure(figsize=(14, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Common Words in Suggestion Reasons', fontsize=18)
plt.tight_layout()
fig.update_layout(
    width=1200,  # wider canvas
    margin=dict(l=50, r=200, t=80, b=50)  # prevent right-side clipping
)

plt.show()
```

Common Words in Suggestion Reasons

account
call
understand
driver
last visit
weeks Schedule
FSL
TRx
week
last
past weeks
Consider planning
preceding week
CGM preference
change
sell FSL
RTE utilizing
weeks Consider
weeks compared
TRx Share
received
opened
claims data
expected transition
moved
Plus Brochure
Unknown
Proven Clinical
make
CGM vs
compared
vs BGM
Oral
newer products
Oral Patients
TRx goals
Clinical Outcomes
plan actions
case study
eDetail Aid
MDI Patient
CGM Naive
Naive FSL
Commercial FSL
Schedule
Meet
use
best patient
behind HCP
Third Party
FSL value
Debra Patient
coverage quality
expected prescriptions
FSL Plus
CI Patient
reinforce
reverse trend
last call
FSL prescriptions
Accessibility Affordable
BGM vs
spoke
fulfilment
Validation Guidelines
HCP TRx
Debra case
FreeStyle Libre
Consider following
patient pathway
Aid BGM
Aggregated Commercial
staff
FSL3 Debra
CGM page
Libre systems
total
HCP
ADC
Party Validation
SalesAssist RTE
TRx volume
Affordable Consider
using FSL
discussing CGM
Patient Case
FSL coverage
offices realize
Outcomes leveraging
work
vs CGM
slower
discussing Proven
month Schedule
value using
MDI
complete month
CGM Selective
CGM Industry
last complete
communicate
increased
sensor page
Dr
increase FSL
Access Accessibility
Industry Guidelines
Plus Sensor
Plus Ends
Aid Value
Aid Clinical
Outcomes Consider
quality across
Guidelines leveraging
empower patients
increase
leave behind
RTE HCP
RTE
N
explain
Debra case
Time
Access Affordability
Overview RTE
achieve better
CGM email
FSL system
drove
Aid Value
Range Improvement
Integration Calculator
glycemic status
selection FSL
Portfolio ABC
Sensor MDI
Aid Value
Share
Selection Tool
Outcomes section
utilizing Access
Outcomes FSL
Average
Optional
Differentiated Features
Critical Thinking
Feature Integrations
Success
Good
Proven Outcomes
Patient Outcomes
Prescribing Trage
utilizing Prescribing
tracking supchemo
Debra Patient
coverage quality
expected prescriptions
Libre systems
CGM page
help
Discuss TRx
discussing Third
one page
recently opened
Undo sell
Libre Plus
CGM section
value prop
Guidelines leave
best patient
behind HCP
use
Meet
Intro ABC
BGM vs
utilizing Lifestyle
CGM metrics
Patient Consent
Accessibility Affordable
FSL prescriptions
BGM Leveraging
reverse trend
last call
expected transition
claims data
reinforce FSL
request Biometrics
CI Patient
plan actions
Clinical Outcomes
weeks Consider
Oral Patients
TRx goals
sensor page
communicate
increased
newer products
CGM Selective
complete month
MDI
Tracking behind
RTE utilizing
sell FSL
change
preceding week
CGM preference
Consider planning
slower
discussing Proven
month Schedule
value using
Calculator
Selection FSL
Portfolio ABC
Sensor MDI
Aid Value
Aid Value
Range Improvement
Integration Calculator
glycemic status

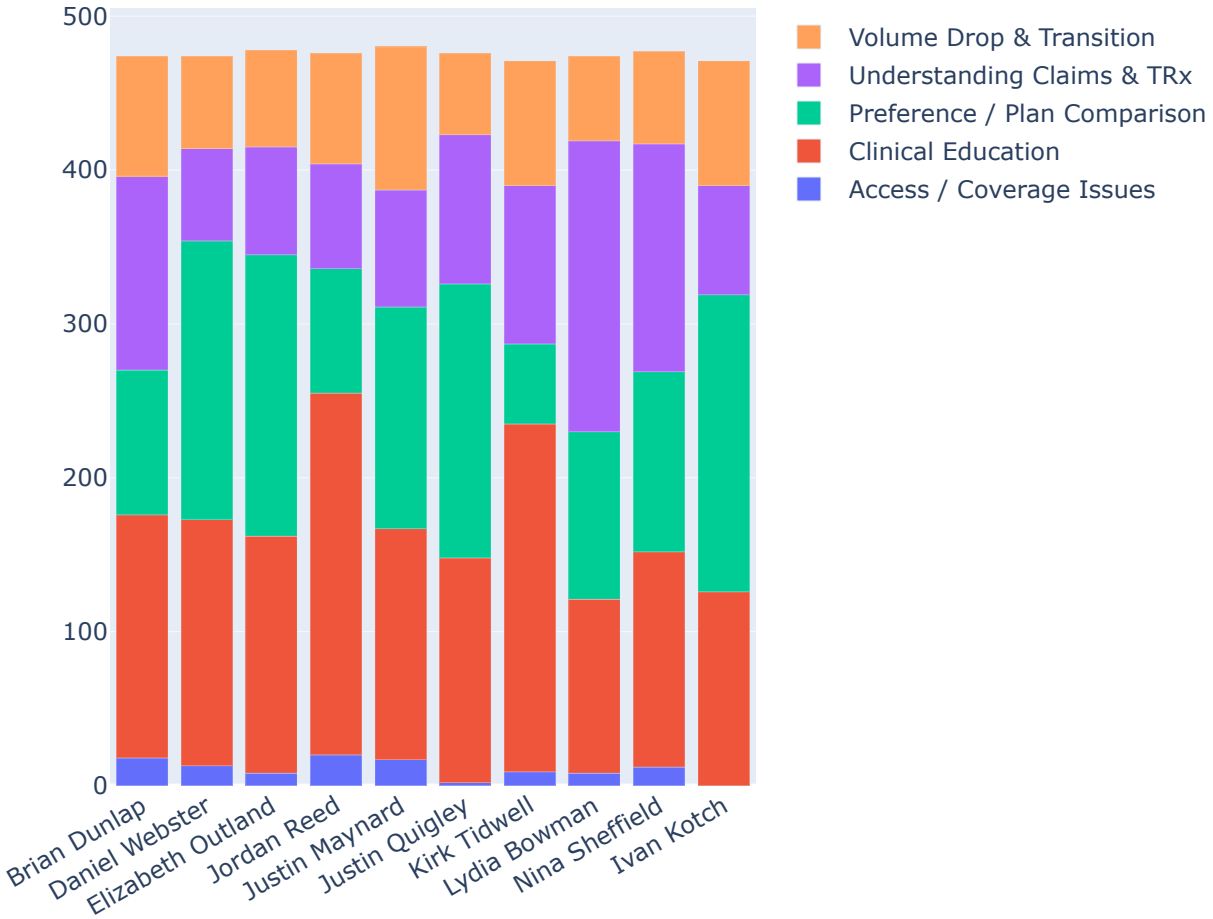
```
# Step 2: TF-IDF vectorization
vectorizer = TfidfVectorizer(stop_words='english', max_features=1000)
X = vectorizer.fit_transform(df_nlp['Reason_clean'])

# Step 3: KMeans clustering (5 themes)
kmeans = KMeans(n_clusters=5, random_state=42, n_init='auto')
df_nlp['Topic_Cluster'] = kmeans.fit_predict(X)


# Extract Top Keywords per Cluster
def get_top_keywords(model, vectorizer, n_terms=10):
    terms = vectorizer.get_feature_names_out()
    keywords = {}
    for i, center in enumerate(model.cluster_centers_):
        top_indices = center.argsort()[-n_terms:][::-1]
        keywords[i] = [terms[j] for j in top_indices]
    return keywords

keywords_by_cluster = get_top_keywords(kmeans, vectorizer)

for cluster, words in keywords_by_cluster.items():
    print(f"Cluster {cluster}: {', '.join(words)}")

# Manually Label Clusters (based on above output)
# Replace numbers with business-relevant themes
cluster_names = {
    0: 'Volume Drop & Transition',
    1: 'Understanding Claims & TRx',
    2: 'Access / Coverage Issues',
    3: 'Clinical Education',
    4: 'Preference / Plan Comparison'
}

df_nlp['Cluster_Theme'] = df_nlp['Topic_Cluster'].map(cluster_names)
```

```
Cluster 0: prescriptions, trx, volume, actions, expected, plan, fsl, understand, schedul
Cluster 1: outcomes, cgm, planning, opened, leveraging, discussing, received, page, plus
Cluster 2: 90499, intro, sensor, libre, differentiated, integrations, features, freestyl
Cluster 3: 13, weeks, fsl, mdi, moved, change, claims, preference, data, patients
Cluster 4: spoke, following, salesassist, utilizing, adc, rte, access, consider, 2025, a
```

```
# Visualize: Owner vs. Cluster Themes

# Step 1: Prepare the data
theme_summary = df_nlp.groupby(['Owner: Full Name', 'Cluster_Theme']) \
    .size().reset_index(name='Count')

top_owners = df_nlp['Owner: Full Name'].value_counts().head(10).index
theme_summary = theme_summary[theme_summary['Owner: Full Name'].isin(top_owners)]

# Pivot data for table
table_df = theme_summary.pivot(index='Owner: Full Name', columns='Cluster_Theme', value
s='Count').fillna(0).astype(int)
table_df = table_df.reset_index()  # For proper column access
```

```python
# Step 2: Create a subplot layout (bar + table)
fig = make_subplots(
    rows=2, cols=1,
    row_heights=[0.6, 0.4],
    vertical_spacing=0.1,
    specs=[[{"type": "xy"}], [{"type": "table"}]]
)

# Step 3: Add stacked bar traces
for theme in theme_summary['Cluster_Theme'].unique():
    theme_data = theme_summary[theme_summary['Cluster_Theme'] == theme]
    fig.add_trace(
        go.Bar(
            x=theme_data['Owner: Full Name'],
            y=theme_data['Count'],
            name=theme
        ),
        row=1, col=1
    )

# Step 4: Add the table below
fig.add_trace(
    go.Table(
        header=dict(
            values=list(table_df.columns),
            fill_color='lightblue',
            align='left'
        ),
        cells=dict(
            values=[table_df[col] for col in table_df.columns],
            fill_color='lavender',
            align='left'
        )
    ),
    row=2, col=1
)

# Step 5: Final layout formatting
fig.update_layout(
    height=900,
    barmode='stack',
    title='Suggestion Themes by Owner + Breakdown Table',
    xaxis_tickangle=-30
)

fig.show()
```

Suggestion Themes by Owner + Breakdown Table



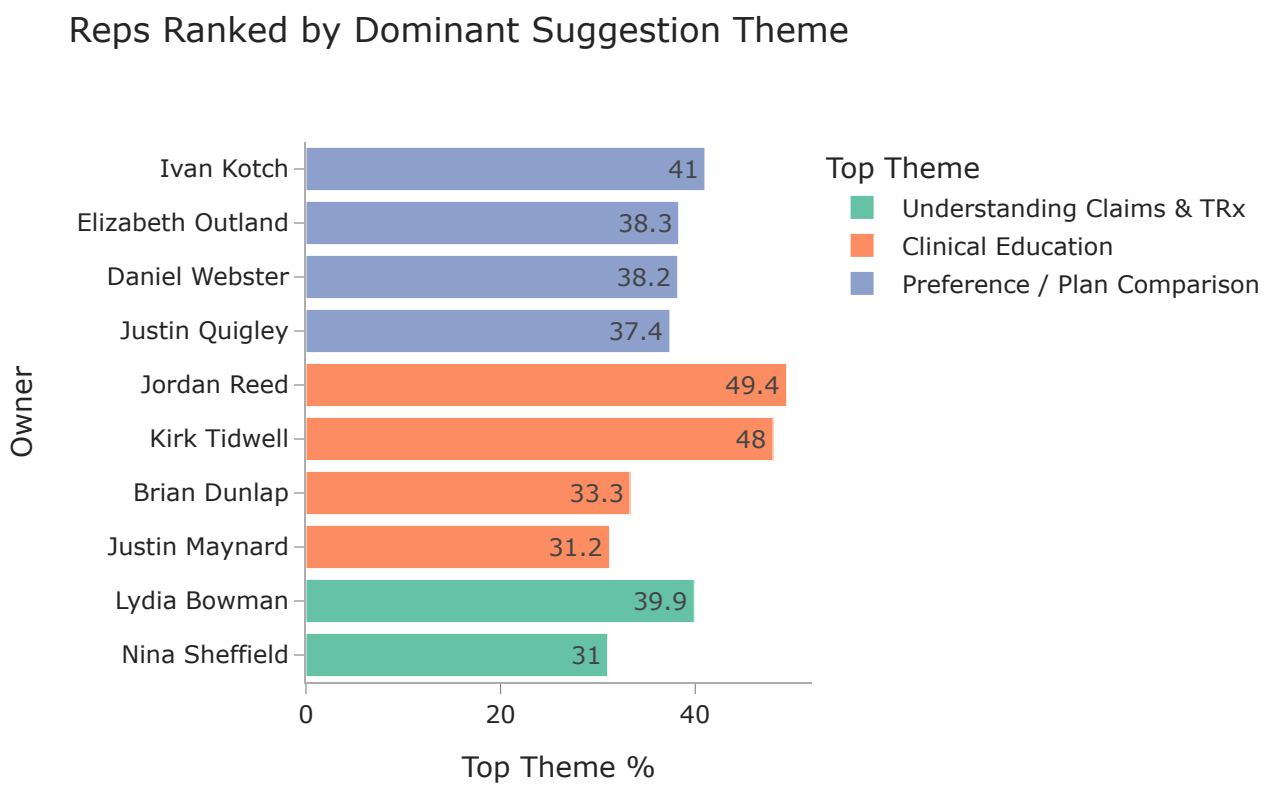
Owner: Full Name	Access / Coverag Issues	Clinical Educati	Preferei / Plan Compar	Underst Claims & TRx	Volume Drop & Transitic
Brian Dunlap	18	158	94	126	78
Daniel Webste	13	160	181	60	60
Elizabet Outland	8	154	183	70	63
Ivan Kotch	0	126	193	71	81

```
# Chart: Reps Ranked by Dominant Suggestion Theme
```

```
# Group and compute % per theme per rep
theme_counts = (
    df_nlp.groupby(['Owner: Full Name', 'Cluster_Theme'])
    .size().reset_index(name='Count')
)

# Compute % contribution of each theme per rep
theme_counts['Total'] = theme_counts.groupby('Owner: Full Name')
['Count'].transform('sum')
theme_counts['Top Theme %'] = (theme_counts['Count'] / theme_counts['Total'] * 100).rou
nd(1)

# Get only the top theme per rep
top_theme_per_rep = (
    theme_counts
    .sort_values(['Owner: Full Name', 'Top Theme %'], ascending=[True, False])
    .drop_duplicates(subset=['Owner: Full Name'], keep='first')
)
```

```
# Owners Grouped by Dominant NLP Theme

import plotly.express as px

# Step 1: Compute counts per owner per theme
theme_counts = (
    df_nlp.groupby(['Owner: Full Name', 'Cluster_Theme'])
    .size().reset_index(name='Count')
)

# Step 2: Compute total suggestions per owner
theme_counts['Total'] = theme_counts.groupby('Owner: Full Name')
['Count'].transform('sum')
theme_counts['Top Theme %'] = (theme_counts['Count'] / theme_counts['Total'] * 100).rou
nd(1)

# Step 3: Filter to top 10 owners by total suggestions
top_owners = theme_counts.groupby('Owner: Full Name')['Count'].sum().nlargest(10).index
theme_counts = theme_counts[theme_counts['Owner: Full Name'].isin(top_owners)]

# Step 4: For each owner, get the top theme
top_theme_per_rep = (
    theme_counts
    .sort_values(['Owner: Full Name', 'Top Theme %'], ascending=[True, False])
    .drop_duplicates(subset=['Owner: Full Name'], keep='first')
)

# Step 5: Create horizontal bar chart
fig = px.bar(
    top_theme_per_rep.sort_values('Top Theme %', ascending=True),
    x='Top Theme %', y='Owner: Full Name',
    color='Cluster_Theme',
    orientation='h',
    text='Top Theme %',
    title='Reps Ranked by Dominant Suggestion Theme',
```

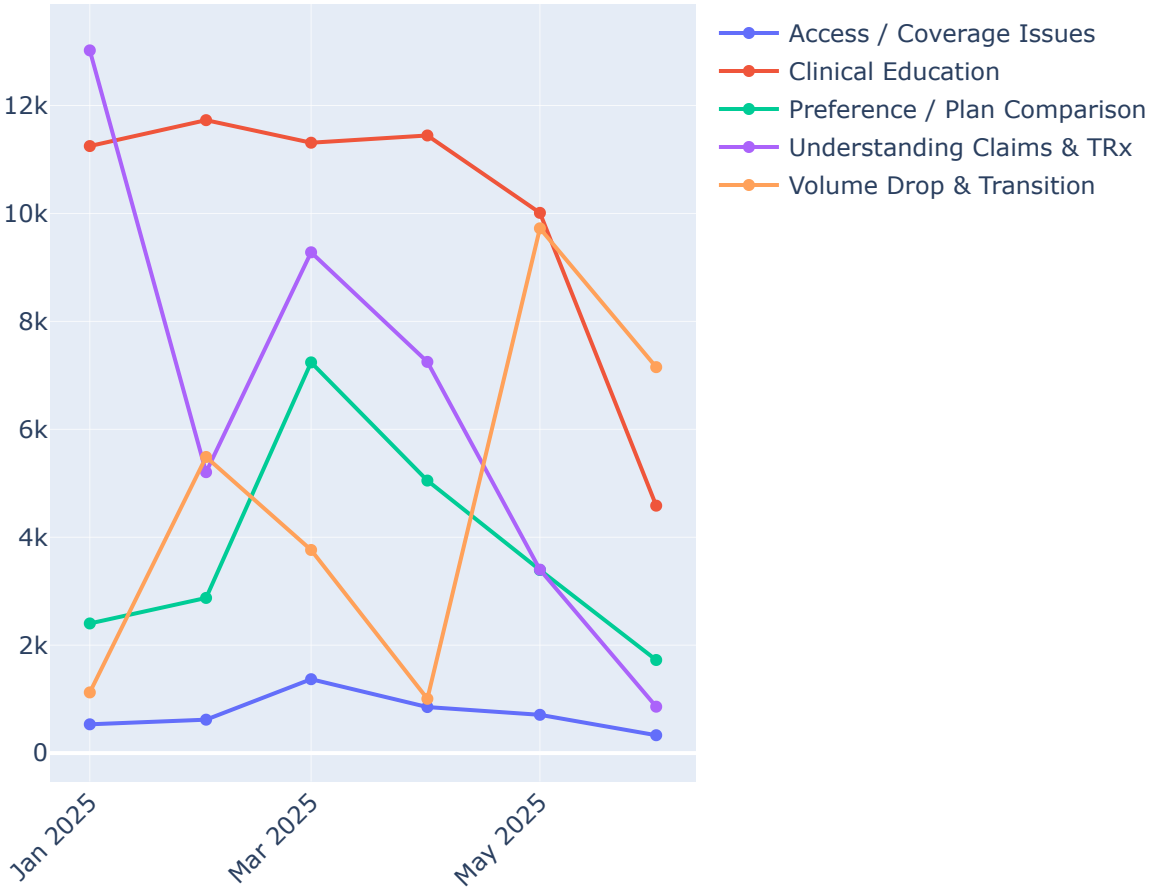
```
    labels={
        'Owner: Full Name': 'Owner',
        'Cluster_Theme': 'Top Theme',
        'Top Theme %': 'Top Theme %'
    },
    template='simple_white',
    color_discrete_sequence=px.colors.qualitative.Set2
)

fig.show()
```




```
# Use Case: Track Emerging Topics Over Time

import plotly.graph_objects as go
from plotly.subplots import make_subplots
import plotly.express as px

# Step 1: Prepare data
df_nlp['Posted Month'] = df_nlp['Posted Date'].dt.to_period('M').astype(str)

monthly_theme = (
    df_nlp.groupby(['Posted Month', 'Cluster_Theme'])
    .size().reset_index(name='Suggestion Count')
)

# Pivot for table format
pivot_df = monthly_theme.pivot(index='Cluster_Theme', columns='Posted Month', values='S
uggestion Count') \
```

```
    .fillna(0).astype(int).reset_index()

# Step 2: Create subplot with 2 rows (line + table)
fig = make_subplots(
    rows=2, cols=1,
    shared_xaxes=False,
    row_heights=[0.6, 0.4],
    vertical_spacing=0.1,
    specs=[[{"type": "xy"}], [{"type": "table"}]]
)

# Step 3: Add line traces for each theme
for theme in monthly_theme['Cluster_Theme'].unique():
    theme_data = monthly_theme[monthly_theme['Cluster_Theme'] == theme]
    fig.add_trace(
        go.Scatter(
            x=theme_data['Posted Month'],
            y=theme_data['Suggestion Count'],
            mode='lines+markers',
            name=theme
        ),
        row=1, col=1
    )

# Step 4: Add table
fig.add_trace(
    go.Table(
        header=dict(
            values=list(pivot_df.columns),
            fill_color='lightblue',
            align='left'
        ),
        cells=dict(
            values=[pivot_df[col] for col in pivot_df.columns],
            fill_color='lavender',
            align='left'
        )
    ),
    row=2, col=1
)

# Step 5: Final layout
fig.update_layout(
    height=900,
    title='NLP Theme Trend Over Time (Monthly) + Theme Table',
    xaxis_tickangle=-45
)

fig.show()
```

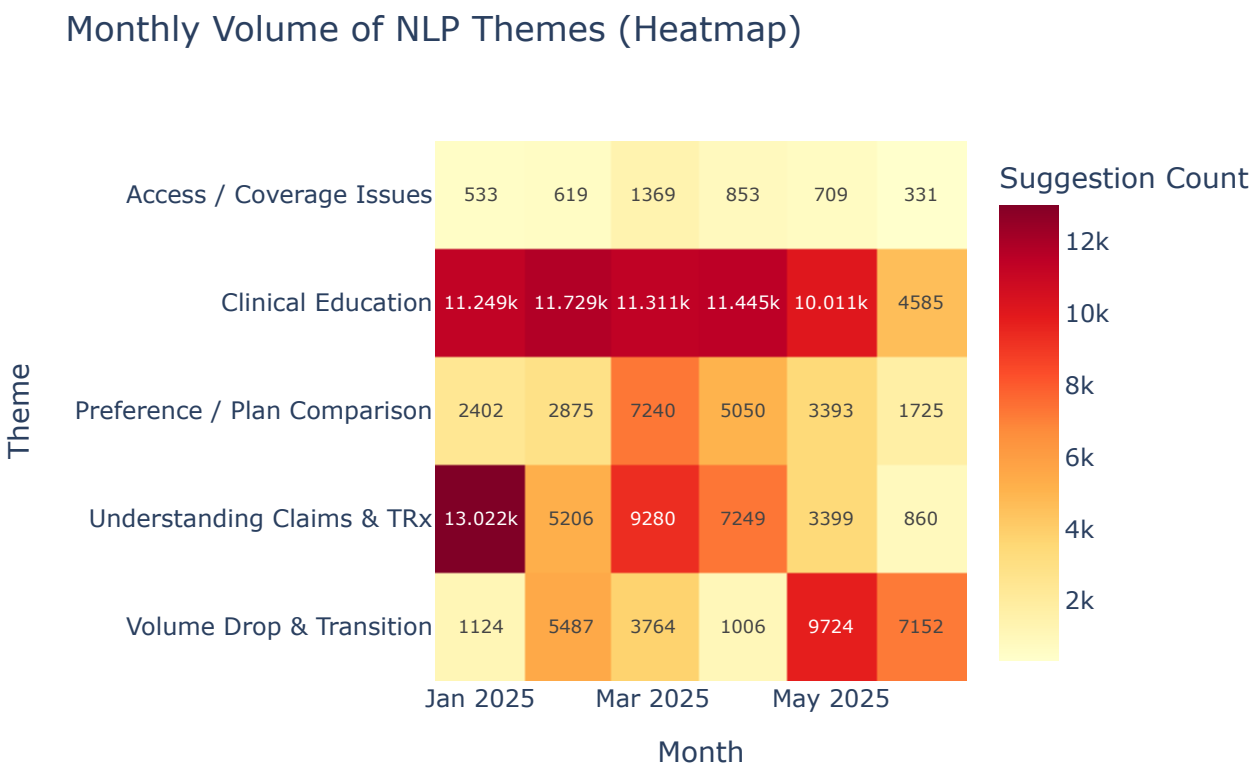
NLP Theme Trend Over Time (Monthly) + Theme Table



Cluste	2025-(	2025-(	2025-(	2025-(	2025-(	2025-(
Access / Covera Issues	533	619	1369	853	709	331
Clinica Educat	11249	11729	11311	11445	10011	4585
Prefere / Plan Compa	2402	2875	7240	5050	3393	1725
Unders	13022	5206	9280	7249	3399	860

```
# Pivot to wide format for heatmap
heatmap_df = monthly_theme.pivot(index='Cluster_Theme', columns='Posted Month', values=
```

```
'Suggestion Count').fillna(0)

# Reset for plotting
heatmap_df = heatmap_df.reset_index().melt(id_vars='Cluster_Theme', var_name='Month', v
alue_name='Count')

# Create heatmap
fig = px.imshow(
    heatmap_df.pivot(index='Cluster_Theme', columns='Month', values='Count'),
    text_auto=True,
    color_continuous_scale='YlOrRd',
    labels=dict(x="Month", y="Theme", color="Suggestion Count"),
    title='Monthly Volume of NLP Themes (Heatmap)'
)


fig.show()
```



Requested Plots and Analysis

```
df.head()
```

	Record Type	Posted Date	Expiration Date	Actioned	Dismissed	Marked As Complete	Title	Reason	Last Modified Date	Last Modified By: Full Name	Owner: Full Name	Accou Name
0	Schedule Call	2025-04-06	2025-04-12	1.0	NaN	NaN	High CGM Growth	<ul><li>Adriana Cadena-Martinez's CGM TRx has ...	2025-04-13	Aktana Integration	Abigail Rosales	Adrian Caden Martin
1	Schedule Call	2025-02-23	2025-03-01	1.0	NaN	NaN	Leading FSL TRx Growth with respect to Goal	<ul><li>Adriana Cadena-Martinez's FSL TRx are ...	2025-03-02	Aktana Integration	Abigail Rosales	Adrian Caden Martin
2	Schedule Call	2025-02-02	2025-02-08	1.0	NaN	NaN	HCP Dexcom Adoption for Conventional Insulin (...	<ul><li>Adriana Cadena-Martinez?s FSL TRx Shar...	2025-02-09	Aktana Integration	Abigail Rosales	Adrian Caden Martin
3	Schedule Call	2025-05-25	2025-05-31	1.0	NaN	NaN	Lagging Product Transition	<ul><li>Alberto Almeida's FSL 2+ and 3+ TRx vo...	2025-06-01	Aktana Integration	Abigail Rosales	Alberto Almeid
4	Schedule Call	2025-01-12	2025-01-18	1.0	NaN	NaN	HCP scripts are not being filled	<ul><li>Alberto Almeida's new patients have a ...	2025-01-19	Aktana Integration	Abigail Rosales	Alberto Almeid

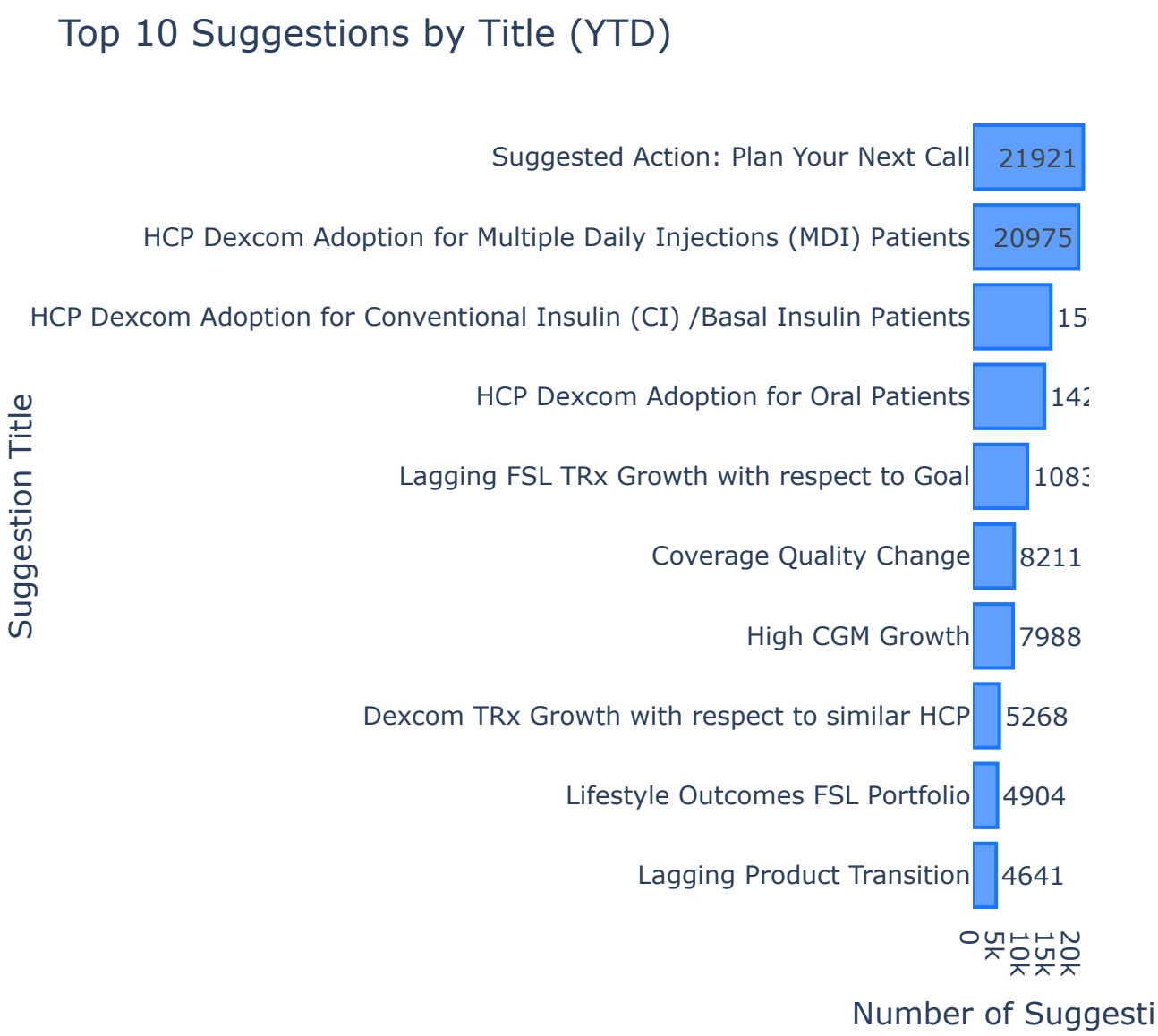
```
# Clean HTML from the Reason Field
from bs4 import BeautifulSoup

df['Reason'] = df['Reason'].apply(lambda x: BeautifulSoup(x, "html.parser").get_text()
if isinstance(x, str) else "")
```

```
# Clean columns
df.columns = df.columns.str.strip()
df['Title'] = df['Title'].astype(str).str.strip()
df['Dismissed'] = df['Dismissed'].astype(str).str.lower()
df['Marked As Complete'] = df['Marked As Complete'].astype(str).str.lower()
df['Actioned'] = df['Actioned'].astype(str).str.lower()
```

```
import plotly.express as px
import plotly.graph_objects as go
```

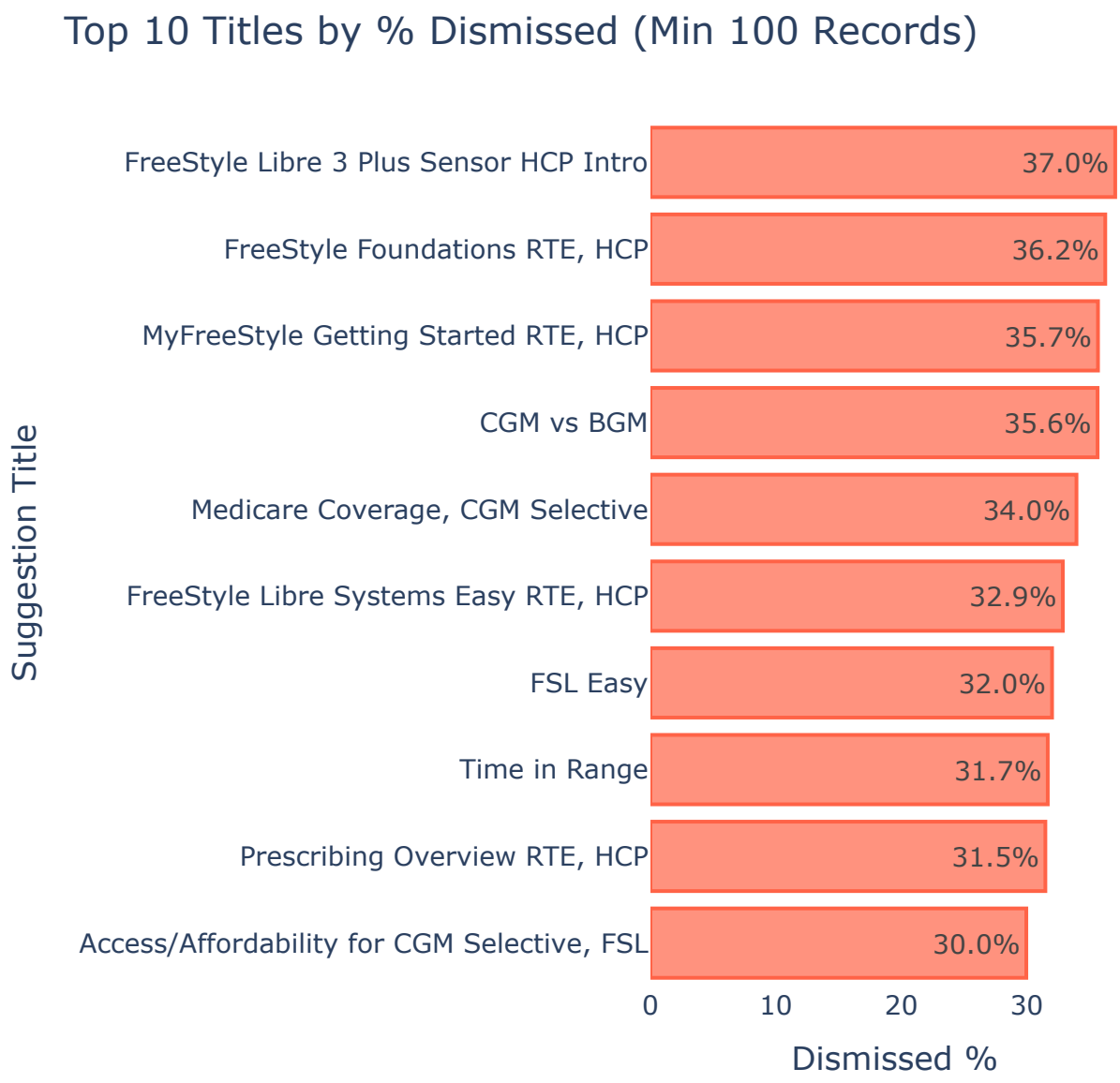
```
# 1. Top 10 Suggestions by Title
# Sort data in ascending order for horizontal bar chart
```

```
top_titles_sorted = (
    df['Title']
      .value_counts()
      .nlargest(10)       # get your top 10
      .sort_values()      # sort for the horizontal bar
)

fig1 = go.Figure()
fig1.add_trace(go.Bar(
    x=top_titles_sorted.values,
    y=top_titles_sorted.index,
    orientation='h',
    marker=dict(
      color='rgba(26,118,255,0.7)',
      line=dict(color='rgba(26,118,255,1.0)', width=2)
    ),
    text=top_titles_sorted.values,
    textposition='auto',
    hoverinfo='x+y'
))
fig1.update_layout(
    title='Top 10 Suggestions by Title (YTD)',
    xaxis_title='Number of Suggestions',
    yaxis_title='Suggestion Title',
    font=dict(size=14),
    height=600,
    plot_bgcolor='white',
    margin=dict(l=200, r=40, t=80, b=40)
)
fig1.show()
```



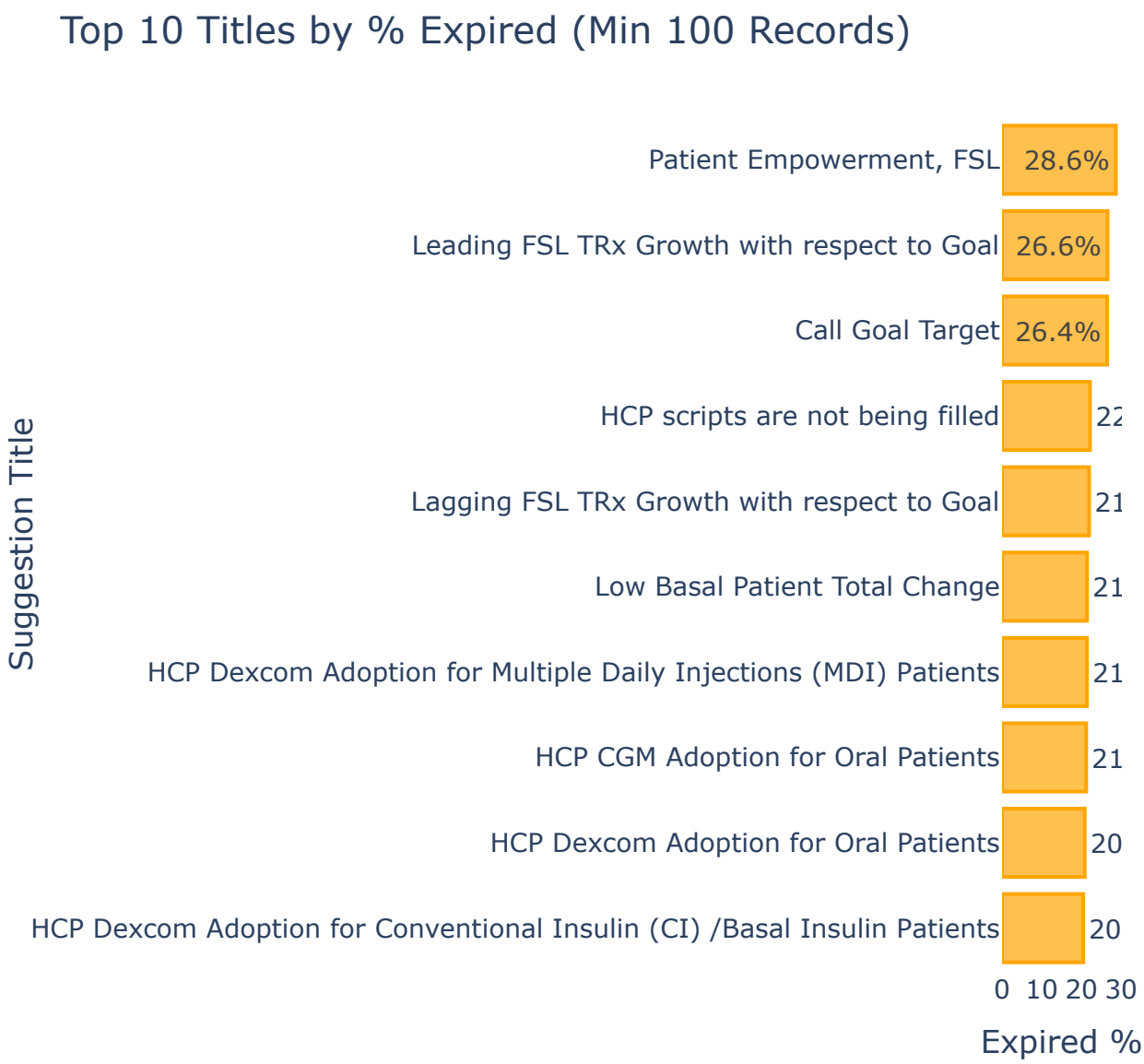

```
# 2. Top 10 by % Dismissed
# Compute % dismissed (confirmed fix for string '1.0')
dismissed_group = df.groupby('Title').agg(
    total=('Title', 'count'),
    dismissed=('Dismissed', lambda x: (x == '1.0').sum())
)
dismissed_group['dismissed_pct'] = 100 * dismissed_group['dismissed'] / dismissed_group
['total']

# Filter and sort
dismissed_filtered = dismissed_group[dismissed_group['total'] >= 100]
top_dismissed = dismissed_filtered.sort_values('dismissed_pct', ascending=False).head(1
0)
top_dismissed_sorted = top_dismissed.sort_values('dismissed_pct')

# Fancy Plot
fig2 = go.Figure()

fig2.add_trace(go.Bar(
```

```
    x=top_dismissed_sorted['dismissed_pct'],
    y=top_dismissed_sorted.index,
    orientation='h',
    marker=dict(color='rgba(255, 99, 71, 0.7)',
                line=dict(color='rgba(255, 99, 71, 1.0)', width=2)),
    text=top_dismissed_sorted['dismissed_pct'].round(1).astype(str) + '%',
    textposition='auto',
    hoverinfo='x+y'
))

fig2.update_layout(
    title='Top 10 Titles by % Dismissed (Min 100 Records)',
    xaxis_title='Dismissed %',
    yaxis_title='Suggestion Title',
    font=dict(size=14),
    height=600,
    plot_bgcolor='white',
    margin=dict(l=200, r=40, t=80, b=40)
)

fig2.show()
```




```
# 3. Top 10 by % Expired

# Define expired rows where none of the fields are '1.0'
expired_df = df[
    (df['Dismissed'] != '1.0') &
    (df['Marked As Complete'] != '1.0') &
    (df['Actioned'] != '1.0')
]

# Aggregate counts
expired_group = expired_df.groupby('Title').size().rename('expired_count')
total_group = df.groupby('Title').size().rename('total_count')

# Combine and calculate %
combined = pd.concat([expired_group, total_group], axis=1).dropna()
combined = combined[combined['total_count'] >= 100]
combined['expired_pct'] = 100 * combined['expired_count'] / combined['total_count']
top_expired = combined.sort_values('expired_pct', ascending=False).head(10)
top_expired_sorted = top_expired.sort_values('expired_pct')
```

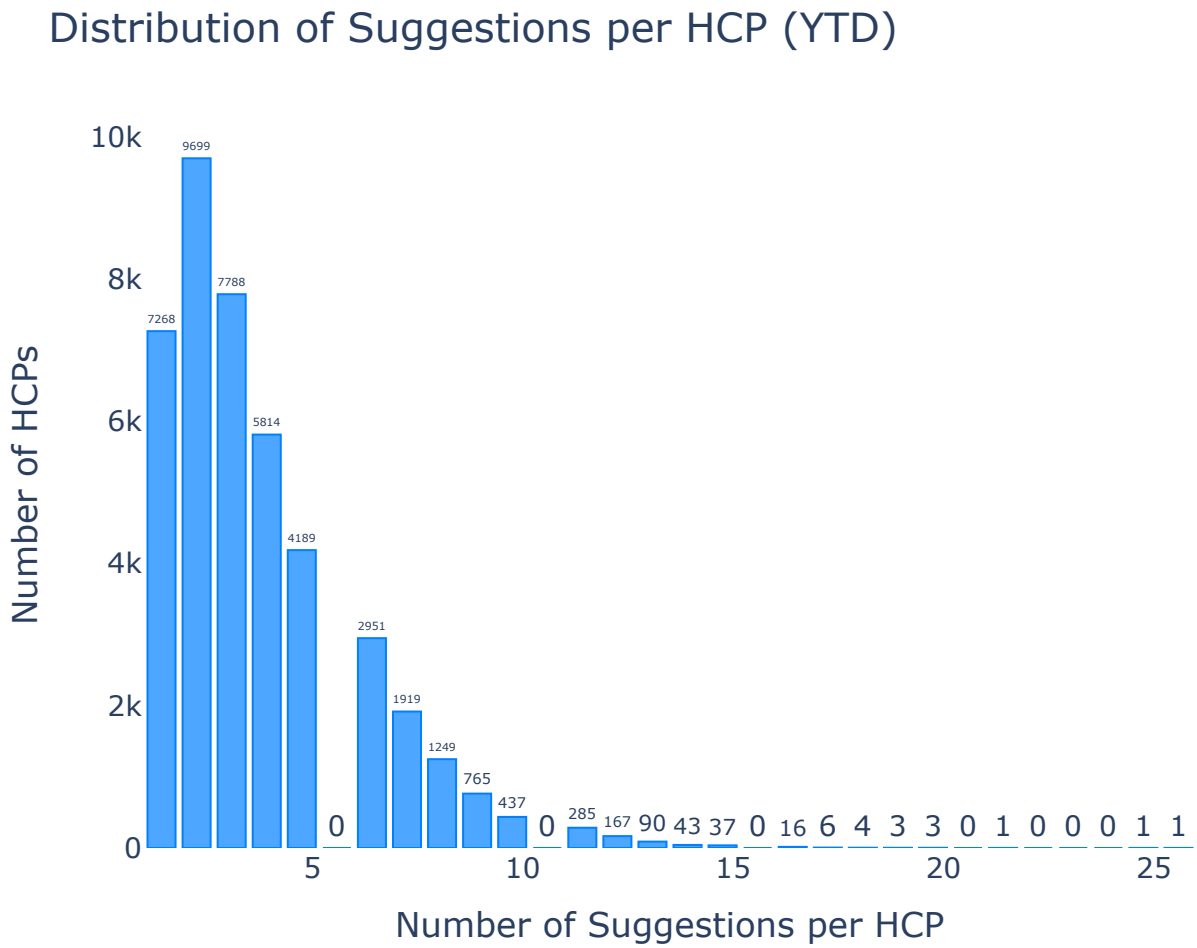
```
# Plot
fig3 = go.Figure()

fig3.add_trace(go.Bar(
    x=top_expired_sorted['expired_pct'],
    y=top_expired_sorted.index,
    orientation='h',
    marker=dict(color='rgba(255, 165, 0, 0.7)',  # orange tone
                line=dict(color='rgba(255, 165, 0, 1.0)', width=2)),
    text=top_expired_sorted['expired_pct'].round(1).astype(str) + '%',
    textposition='auto',
    hoverinfo='x+y'
))

fig3.update_layout(
    title='Top 10 Titles by % Expired (Min 100 Records)',
    xaxis_title='Expired %',
    yaxis_title='Suggestion Title',
    font=dict(size=14),
    height=600,
    plot_bgcolor='white',
    margin=dict(l=200, r=40, t=80, b=40)
)

fig3.show()
# Get count of suggestions per HCP
hcp_counts = df['Account: Name'].value_counts()
```




```python
import numpy as np
```

```python
# 4. Distribution of Suggestions per HCP
# Count suggestions per HCP
hcp_values = df['Account: Name'].value_counts()

# Create histogram bins
counts, bins = np.histogram(hcp_values, bins=30)
bin_centers = 0.5 * (bins[:-1] + bins[1:])

# Fancy labeled histogram
fig4 = go.Figure()
fig4.add_trace(go.Bar(
    x=bin_centers,
    y=counts,
    text=counts,
    textposition='outside',
    marker=dict(color='rgba(0, 128, 255, 0.7)',
                line=dict(color='rgba(0, 128, 255, 1.0)', width=1))
```

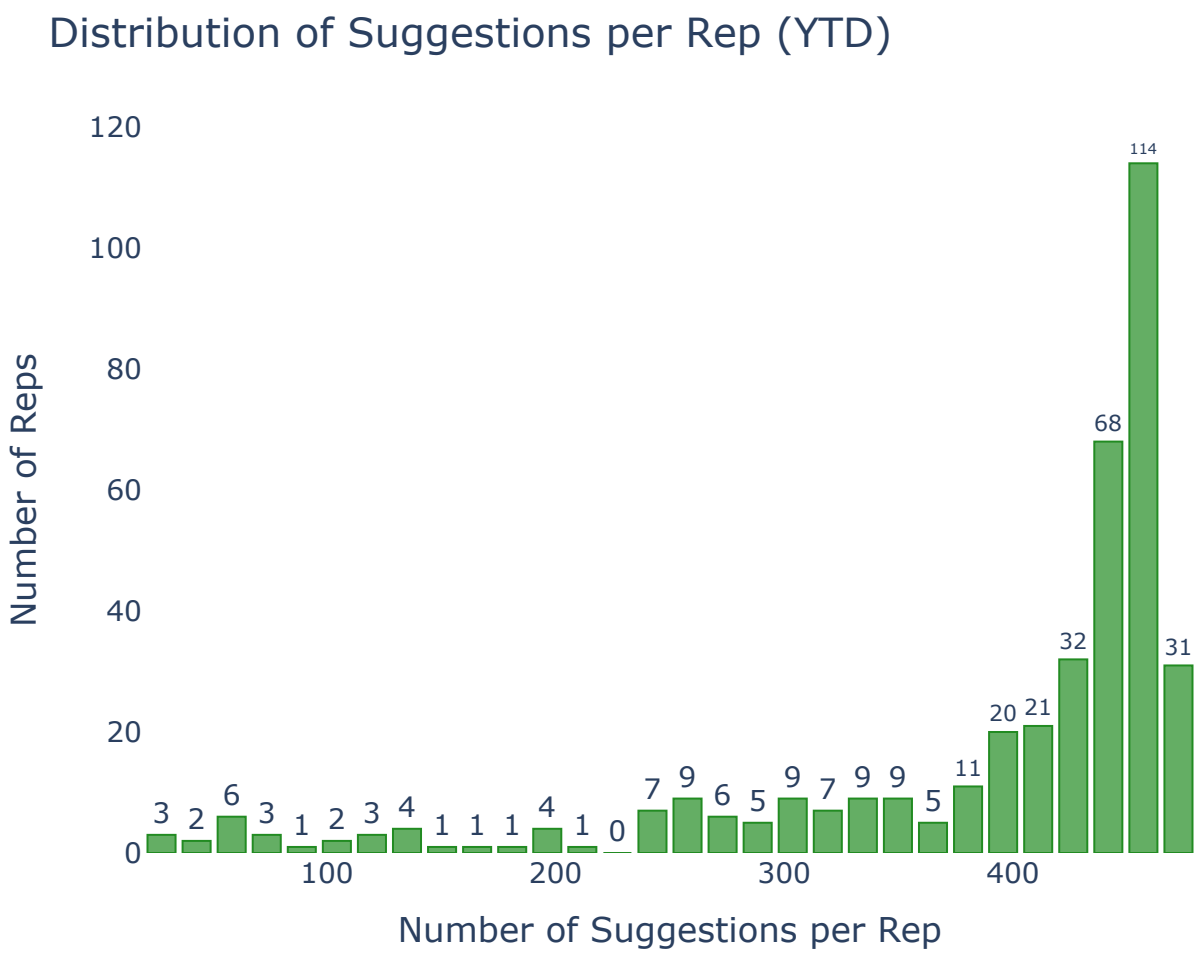
```
))
fig4.update_layout(
    title='Distribution of Suggestions per HCP (YTD)',
    xaxis_title='Number of Suggestions per HCP',
    yaxis_title='Number of HCPs',
    font=dict(size=14),
    height=500,
    plot_bgcolor='white',
    margin=dict(l=80, r=40, t=80, b=40)
)
fig4.show()

# Create readable summary table for ranges
bucket_edges = [0, 2, 5, 10, 15, float('inf')]
bucket_labels = ['1-2', '3-5', '6-10', '11-15', '>15']
bucket_counts = pd.cut(hcp_values, bins=bucket_edges, labels=bucket_labels, include_low
est=True).value_counts().sort_index()
percentages = (bucket_counts.values / len(hcp_values) * 100).round(1)

# Safely format the table
summary_df = pd.DataFrame({
    'Range of Suggestions per HCP': bucket_labels,
    'Number of HCPs': bucket_counts.values,
    '% of Total HCPs': [f"{p:.1f}%" for p in percentages]
})

summary_df
```



	Range of Suggestions per HCP	Number of HCPs	% of Total HCPs
0	1–2	16967	39.7%
1	3–5	17791	41.6%
2	6–10	7321	17.1%
3	11–15	622	1.5%
4	>15	35	0.1%

```
# 5. Distribution of Suggestions per Rep

# Get count of suggestions per Rep
rep_values = df['Owner: Full Name'].value_counts()

# Create histogram bins
rep_counts, rep_bins = np.histogram(rep_values, bins=30)
rep_bin_centers = 0.5 * (rep_bins[:-1] + rep_bins[1:])

# Create labeled histogram-style bar chart
fig5 = go.Figure()

fig5.add_trace(go.Bar(
    x=rep_bin_centers,
    y=rep_counts,
```

```
    text=rep_counts,
    textposition='outside',
    marker=dict(color='rgba(34, 139, 34, 0.7)',  # green tone
                line=dict(color='rgba(34, 139, 34, 1.0)', width=1))
))

fig5.update_layout(
    title='Distribution of Suggestions per Rep (YTD)',
    xaxis_title='Number of Suggestions per Rep',
    yaxis_title='Number of Reps',
    font=dict(size=14),
    height=500,
    plot_bgcolor='white',
    margin=dict(l=80, r=40, t=80, b=40)
)

fig5.show()

# ----------------------------
# Summary Table Below the Chart
# ----------------------------

# Define custom buckets
bucket_edges = [0, 50, 100, 200, 300, float('inf')]
bucket_labels = ['1-50', '51-100', '101-200', '201-300', '>300']

# Bin reps by number of suggestions
bucket_counts = pd.cut(rep_values, bins=bucket_edges, labels=bucket_labels, include_low
est=True).value_counts().sort_index()
percentages = (bucket_counts.values / len(rep_values) * 100).round(1)

# Format table
rep_summary_df = pd.DataFrame({
    'Range of Suggestions per Rep': bucket_labels,
    'Number of Reps': bucket_counts.values,
    '% of Total Reps': [f"{p:.1f}%" for p in percentages]
})

# Display table
rep_summary_df
```



	Range of Suggestions per Rep	Number of Reps	% of Total Reps
0	1–50	5	1.3%
1	51–100	11	2.8%
2	101–200	12	3.0%
3	201–300	33	8.4%
4	>300	334	84.6%